내공 있는 프로그래머로 길러주는

컴파일러의 이해

Chapter 11
병렬성과 지역성을 위한 최적화

목차

01 병렬 처리

02 자료 종속과 자료 종속 분석

03 루프 변환

학습목표

- 병렬 처리 기본 개념에 대해 이해할 수 있다.
- 자료 종속과 자료 종속 분석에 대해 이해할 수 있다.
- 루프 융합, 루프 분리, 루프 역순, 루프 교환, 루프 기울임, 스트립 마이닝, 루프 타일링 등 루프 변환 방법에 대해 이해할 수 있다.

11.1 병렬 처리

IT COOKBOOK

- **병렬 처리가 필요성** : 빅데이터, 사물 인터넷(internet of things, IoT) 등의 정보 기술 이슈가 대두된 스마트 시대에는 아주 많은 자료를 빠른 시간에 처리하기 위함.
- **병렬 처리가 필요한 예** : 일기예보나 DNA 구조를 밝히기 위한 모델링과 시뮬레이션, 항공 추진 시스템을 설계하기 위한 유체 동역학(fluid-dynamics)
- **병렬성과 지역성을 높이는 가장 기본적인 컴파일러 기법** : 일반적으로 프로세서가 어떤 프로그램을 실행하면서 최근 사용한 자료를 자주 사용한다면 그 프로그램의 자료 지역성(data locality)이 우수하다고 한다. 다중 프로세서 시스템에서 어떤 프로세서의 자료 지역성이 높으면 당연히 그 프로세서는 다른 프로세서와 자주 통신할 필요가 없어지므로 병렬 처리 시스템의 효율을 높일 수 있다.
- **병렬성을 높게 하기 위한 방법** : 프로그램을 분할해야 하지만 일반 프로그램을 분할한다고 해서 여러 개의 분할을 동시에 수행할 수 있는 것은 아니다. 프로그램을 분할하려면 각 자료 사이의 종속성을 계산할 수 있어야 한다. 계산을 통해 해당 문장을 여러 프로세서에서 동시에 수행해도 되는지 결정할 수 있다.

11.1 병렬 처리

IT COOKBOOK

- 다음 프로그램을 살펴보자.

```
for I = 1 to 100 do
  A(I) = B(I) * C(I)
  B(I) = C(I) / D(I)
Endfor
```

- 이 프로그램에서는 루프가 반복될 때 값이 정의(define)되거나 사용(use)되는 위치가 다르기 때문에 서로 다른 프로세서에서 루프의 반복을 병렬로 수행할 수 있다.
- p1, p2, ..., p100 등 프로세서가 100개 있다고 가정하자. 그러면 100개의 프로세서에서 다음과 같이 병렬로 수행될 수 있다.

11.1 병렬 처리

IT COOKBOOK

p1 : A(1) = B(1) * C(1)

B(1) = C(1) / D(1)

p2 : A(2) = B(2) * C(2)

B(2) = C(2) / D(2)

⋮

p100 : A(100) = B(100) * C(100)

B(100) = C(100) / D(100)

11.1 병렬 처리

IT COOKBOOK

- 또 다른 프로그램을 살펴보자.

```
for I = 1 to 100 do
    A(I) = B(I) * C(I)
    B(I) = C(I) / A(I-3)
endfor
```

- 같은 변수인 A를 첫 번째 문장에서는 A(I)로 정의하고 두 번째 문장에서는 A(I-3)으로 사용하므로 이 두 문장을 동시에 처리하면 문제가 생긴다. 이와 같은 경우를 자료 종속성(data dependence)이 존재한다고 말한다.
- 자료 종속성이 존재하는 경우의 문장들은 순차적으로 실행 해야 하지만, 종속성이 없는 문장과 종속성이 있는 문장이 혼재되어 있는 프로그램에서는 문장이 어떤 순서로 어떻게 실행되는지에 따라 병렬성을 높일 수 있으므로, 루프가 몇 번째 반 복될 때 같은 메모리 위치를 가리킬 가능성이 있는지 판단하는 것이 중요하다.
- 이 정보를 알아야 다중 프로세서에서 명령어 실행 순서를 결정할 수 있는데, 이처럼 실행 순서를 결정하는 것을 코드 스케줄링이라 한다. 즉 모든 자료의 종속성을 준수함과 동시에 같은 위치를 참조하는 연산을 같은 프로세서에서 모아 실행하면 효율이 좋아진다.

11.1 병렬 처리

IT COOKBOOK

- 다른 예를 하나 더 보자.

```
for i=1 to 5 do
    x(i) = x(i-1) + y(i-1)
    y(i) = x(i-2) - y(i-1)
endfor
```

$P_1(i)$

```
s(i) = x(i) + y(i)
if s(i) ≥ 100
    then a = a+1
else b = b+1
```

$P_2(i)$

- 이 프로그램은 P1(i)와 P2(i)라는 2개의 프로세스로 나눌 수 있다. 그런데 이 경우에는 두 프로세스를 병렬로 처리할 수 없다. P1(i)에서 계산되는 x(i)와 y(i) 값을 P2(i)에서 다시 사용하기 때문이다. 하지만 순차적으로 실행해보면 다음과 같이 동시에 부분적으로 처리할 수 있다

11.1 병렬 처리

IT COOKBOOK

x(1) = x(0) + y(0)

y(1) = x(-1) - y(0)

==========

s(1) = x(1) + y(1)

x(2) = x(1) + y(1)

y(2) = x(0) - y(1)

==========

s(2) = x(2) + y(2)

x(3) = x(2) + y(2)

y(3) = x(1) - y(2)

==========

s(3) = x(3) + y(3)

x(4) = x(3) + y(3)

y(4) = x(2) - y(3)

==========

s(4) = x(4) + y(4)

x(5) = x(4) + y(4)

y(5) = x(3) - y(4)

==========

s(5) = x(5) + y(5)

11.1 병렬 처리

- 결과적으로 i = 1일 때는 P1만 처리하지만 i = 2부터는 P1과 P2를 병렬로 처리할 수 있다. 이때 P2를 처리하는 프로세서는 P1을 처리하는 프로세서로부터 그 이전에 계산한 x(i-1)과 y(i-1)을 전송 받아 사용하기 때문에 프로세서 간 통신(inter-processor communication)에 따른 오버헤드가 발생한다. 결국 빠른 실행을 위해서는 선형 대수와 정수 프로그래밍(integer programming)을 기초로 하는 자료 종속을 계산하고, 병렬 코드의 통신을 최소화하기 위해 연관된 연산을 모아서 같은 프로세서에 할당하는 자료 지역성을 높여야 한다.

11.1 병렬 처리

병렬 처리 시스템

- 병렬 처리 시스템은 제어 방법, 형태, 프로세서, 정보 자원의 결합 정도에 따라 분류하는 방법 등이 있다.
- 가장 대표적인 방법은 플린(Michael Flynn)에 의한 분류로, 프로세서들이 처리하는 명령어와 자료의 흐름 수에 따라 다음과 같이 네 가지로 나눌 수 있다.
  - 단일 명령 단일 자료 처리(single instruction stream single data stream, SISD)
  - 단일 명령 다중 자료 처리(single instruction stream multi data stream, SIMD)
  - 다중 명령 단일 자료 처리(multi instruction stream single data stream, MISD)
  - 다중 명령 다중 자료 처리(multi instruction stream multi data stream, MIMD)
- 그리고 프로세서, 기억장치 및 상호 연결 네트워크를 어떻게 구성하느냐에 따라 다음과 같이 구분할 수도 있다.
  - 대칭적 다중 프로세서(symmetric multi-processor, SMP)
  - 대규모 병렬 프로세서(massively parallel processor, MPP)
  - 캐시 일관성 NUMA(cache-coherent nonuniform memory access, CC-NUMA)
  - 분산 시스템(distributed system)
  - 클러스터 컴퓨터(cluster computer)

11.1 병렬 처리

- **대칭적 다중 프로세서**
  - 2~64개 정도의 프로세서로 구성되는 시스템
  - 공유 메모리(shared memory)
  - 프로세서 간 통신은 공유 메모리를 통해 이루어지며, 그 구조는 [그림 11-1]과 같다.

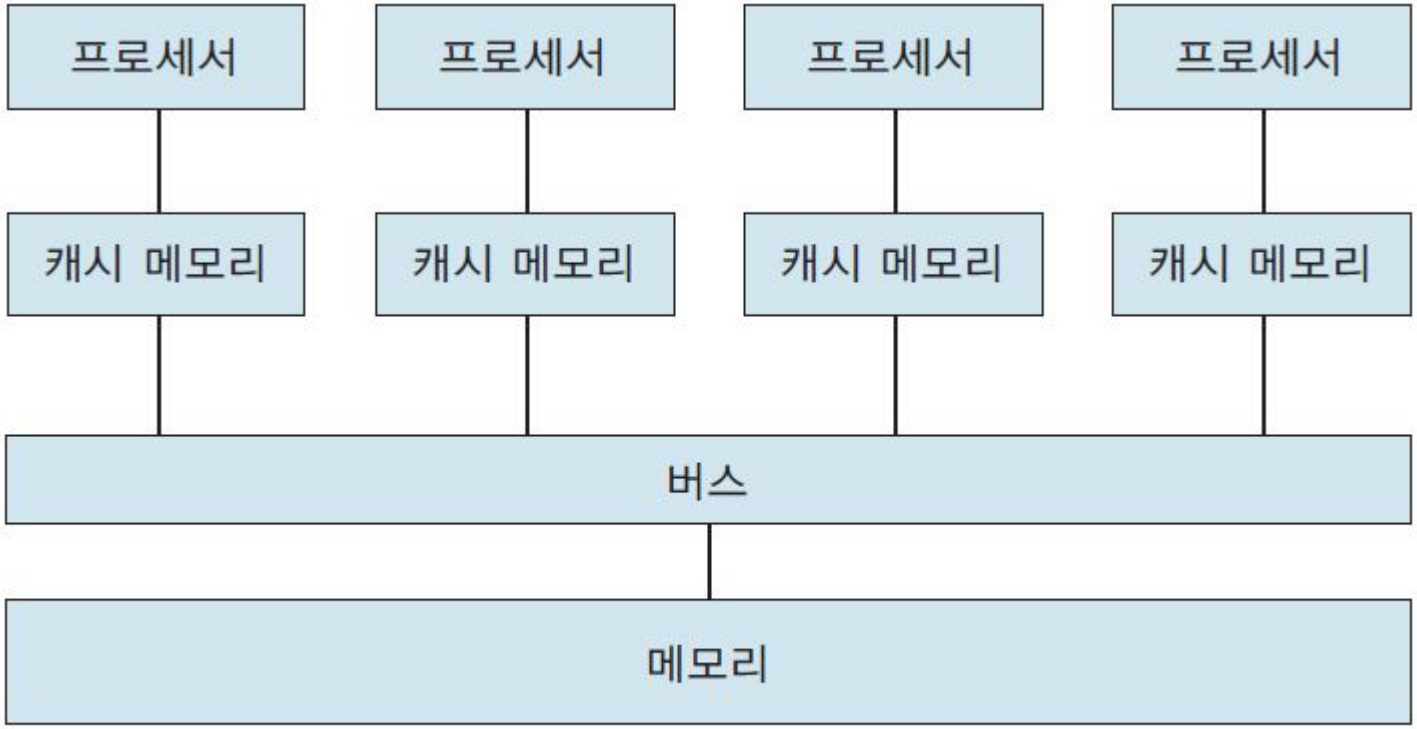


그림 11-1 대칭적 다중 프로세서의 구조

11.1 병렬 처리

IT COOKBOOK

- 대칭적 다중 프로세서는 한 프로세서가 다른 프로세서와 통신하는 시간이 메모리 참조 시간의 2배에 불과하므로 프로세서 간 통신 비용이 상대적으로 작다는 것이 장점
- 하나의 공유 메모리를 여러 개의 프로세서가 공유하여 사용하므로 여러 개 프로세서가 메모리에 동시에 접근하지 못하게 하는 임계 구역(critical section) 처리의 단점

▪ 분산 시스템

- 각 프로세서마다 개별 메모리를 가진 시스템으로 메시지 전달(message passing) 방식을 사용
- 구조는 [그림 11-2]
- 분산 메모리 방식은 각각의 프로세서가 별도의 메모리 공간을 가지고 있기 때문에 프로세서는 서로 메시지를 전달함으로써 통신
- 공유 메모리 방식보다 코드를 작성하기가 더 쉽지만, 메모리의 값이 변하면 다른 메모리의 값을 모두 변경해야 하는, 메시지 전달에 따른 통신 오버헤드가 길다는 단점

11.1 병렬 처리

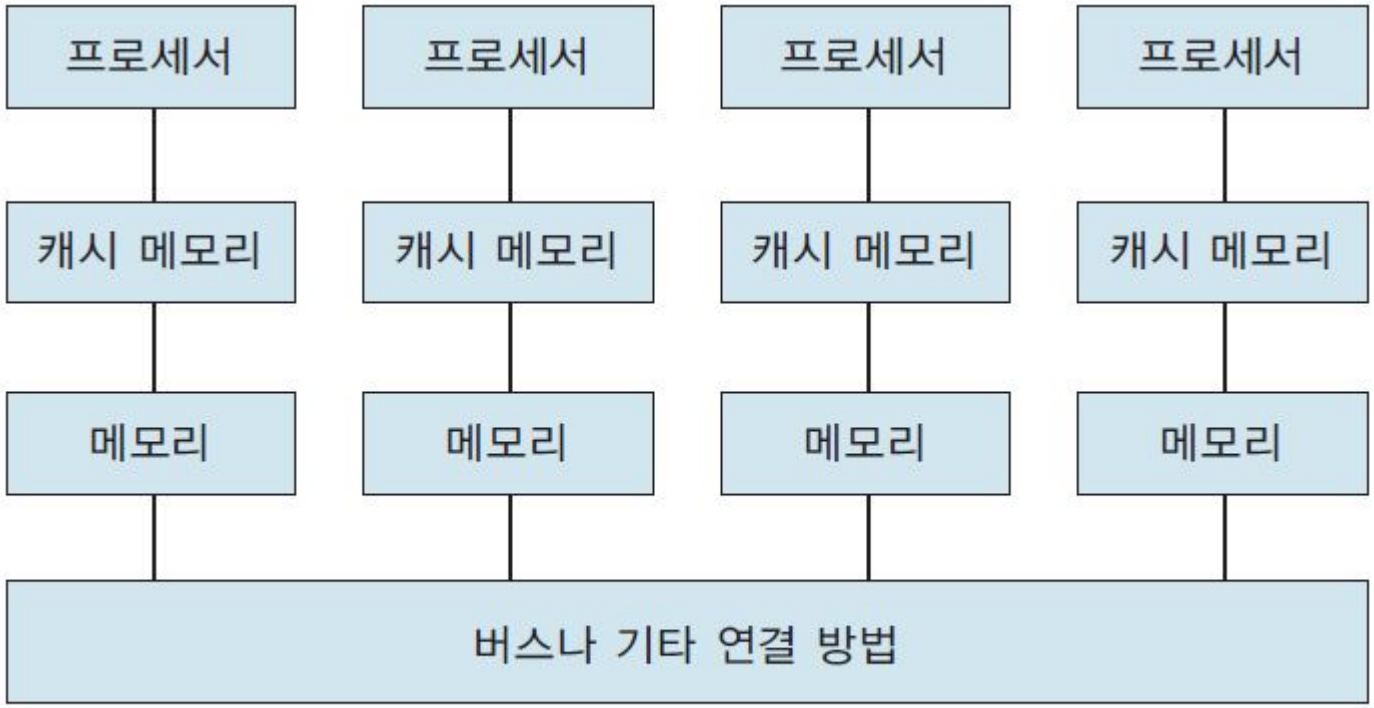


그림 11-2 분산 메모리 시스템의 구조

11.1 병렬 처리

IT COOKBOOK

■ 병렬 처리 시스템에 대한 여러 가지 개념

- 병렬 처리가 나오게 된 것은
  - 단일 프로세서로서의 처리 속도 한계 때문
  - 현재 알려진 단일 프로세서의 최대 계산 능력은 1 GFLOPS(10^9 FLOPS)
  - 하지만 이런 능력으로는 앞에서 언급한 일기예보, DNA 구조 등의 연산이 불가능한데 그 속도 한계를 극복하기 위한 방법이 바로 병렬 처리이다.
  - 여기서는 병렬 처리의 한계, 병렬화 크기, 지역성 등에 대해 간단히 살펴보자
- 병렬 처리 시스템의 문제
  - 병렬 처리의 한계에 대한 여러 가지 이론이 있다.
  - 첫 번째는 민스키(Marvin Minsky)의 모순성이다.
    - 민스키는 p개의 프로세서를 사용한 병렬 컴퓨터에서 프로세서 간의 정보 교환에 따른 통신 오버헤드 때문에 시스템 성능은 p배가 아니라 $\log_2 p$배까지밖에 개선되지 못한다고 주장했다.
    - 그러나 병렬 알고리즘, 상호 연결 네트워크, 스케줄링 등의 기술이 개발되면서 민스키의 모순성에 따른 문제가 많은 부분 개선되었다.
  - 두 번째는 암달의 법칙이다.
    - 암달(Gene Amdahl)은 병렬 처리를 이용하여 얻을 수 있는 속도 향상에 대한 한계를 제시했다. 일반 프로그램에서 병렬화가 가능한 비율을 f라 하고, 통신이나 병렬화 부담 없이 p개의 프로세서로 구성된 병렬 처리 시스템이라면 암달의 법칙에서 속도 향상은 $s_p = 1/((1 - f) + (f / p))$ 이다
      .

IT COOKBOOK

[예제 11-1] 암달의 법칙 계산하기

- 일반 프로그램에서 병렬화가 가능한 부분이 80%이고 8개의 프로세서를 가지고 있는 병렬 시스템의 경우, 암달의 법칙에 의해 어느 정도의 속도 향상을 꾀할 수 있는지 계산해보자.
  - [풀이]

    S_p를 계산. 병렬화 가능한 부분이 80%이므로 f = 0.8임. 프로세서가 8개이므로 p = 8임.

    $\therefore S_p = 1/((1- f) + (f / p))$

    $= 1/((1- 0.8) + (0.8 / 8))$

    $= 10/3$

    $= 3.3$

    결국 3.3배 정도의 속도 향상을 기대할 수 있다.

[예제 11-1]에서 만약 프로세서를 무한으로 늘린다면 f/p가 0으로 수렴하므로 5배 정도의 속도 향상이 기대된다.

병렬화의 크기

- 병렬 처리 시스템에서 사용하는 프로세서의 복잡도
  - 각 프로세서에 할당되는 병렬화 크기(granularity of parallelism)에 따라 결정
  - 병렬성은 병렬화 크기에 따라 미세 크기 병렬화(fine grain parallelism), 중간 크기 병렬화(medium grain parallelism), 큰 크기 병렬화(coarse grain parallelism) 등으로 분류
- 미세 크기 병렬화 - 프로그램을 병렬 처리가 가능한 가장 작은 단위로 분할하는 방법
  - 고급 프로그램 언어의 문장, 어셈블리 명령어, 하나의 산술 논리 연산 단위 크기로 병렬화하는 것
  - 빈번한 자료 교환으로 프로세서 간 통신량이 많이 발생하기 때문에 고속의 상호 연결 네트워크(interconnection network)가 필요
  - 싱킹머신 사의 Connection Machine-1, MIT의 J-Machine, MPP(Massively Parallel Processor) 시스템, STARAN 시스템 및 시스톨릭 배열(systolic array) 등에서 미세 크기 병렬화를 사용
- 큰 크기 병렬화 - 중첩 루프로 구성된 프로그램에서 벡터 또는 행렬 계산을 위해 n개의 내부 루프가 하나의 단위 크기
  - 하나의 프로그램이 벡터나 행렬 계산과 같은 많은 양의 수치 계산을 포함하는 과학 계산에 잘 맞는 단위 크기
  - 고속 벡터 계산을 위한 파이 프라인 산술 연산 장치가 필요하며, 적은 수의 프로세서로 구성된 슈퍼컴퓨터(Cray Y-MP, CRAY-2, NEC SX-6)에서 큰 크기 병렬화를 사용

11.1 병렬 처리

IT COOKBOOK

- 중간 크기 병렬화 - 각 프로세서에 할당되는 단위 작업이 중간 정도 크기
  - 미세 그레인과 큰 그레인의 중간 크기로서, 중첩 루프의 내부 루프나 다중 사용자(multi-user) 시스템의 한 작업(job)이 단위의 크기
  - SGI의 Origin 시스템, 휴렛패커드의 HP 9000 SuperDome, Sun의 UltraSparc, BBN의 Butterfly, 모토로라의 M88000, 인텔의 iPSC, 80×86 계열과 risc 등에서 중간 크기 병렬화를 사용

■ 병렬화 단위 크기와 프로세서 구조의 관계를 전체적으로 살펴보면 –

- 큰 크기 병렬화는 복잡한 연산을 할 수 있으며 컴퓨터 구조도 복잡한 프로세서를 사용
- 미세 크기 병렬화는 간단한 연산만 처리할 수 있으며 컴퓨터 구조도 작고 간단한 프로세서를 사용

■ 병렬 처리 시스템의 효율을 높이려면

- 사용자는 전통적인 방법으로 프로그램을 작성하고, 병렬 처리 시스템 안에 있는 병렬 컴파일러가 병렬 처리 시스템에 맞게 프로그램을 변환하는 것이 가장 좋다.
- 하지만 현재 병렬 컴파일러가 할 수 있는 일은 아주 간단한 수준에서 병렬 처리 시스템에 맞게 프로그램을 변환하는 것이다.
- 그러므로 병렬 처리 시스템을 효율적으로 사용하려면 사용자가 병렬 처리 시스템에 가장 알맞게 작성할 필요가 있다.

11.1 병렬 처리

IT COOKBOOK

- 병렬 처리 시스템에 가장 알맞게 작성하기 위해 하나의 작업을 어느 부분에 대해 어떻게 나누어서 병렬 처리를 할 것인지가 매우 중요한데, 루프는 이런 효과를 가장 잘 나타낼 수 있다.
- 앞에서 다루었던 프로그램을 다시 보자.

```
for I = 1 to 100 do
   A(I) = B(I) * C(I)
   B(I) = C(I) / D(I)
Endfor
```

  - 이 문장에서는 루프가 반복될 때 값이 정의되거나 기록되는 위치가 다르기 때문에 각 루프의 반복을 서로 다른 프로세서에서 병렬로 수행할 수 있다.
    - 결국 프로세서가 100개 존재한다면 100개의 프로세서에서 각 반복에 대해 동시에 수행할 수 있다.
  - 병렬 처리를 더 많이 적용하기 위해 do 문 안에 있는 2개의 문장을 동시에 처리할 수도 있다.
    - 만약 200개의 프로세서가 존재한다면 200개의 문장을 한꺼번에 수행할 수 있을 것이다. 2개의 문장에 대해 검사하고 200개의 문장을 동시에 처리할 수 있으므로 병렬화의 효과가 매우 높아진다.

11.1 병렬 처리

IT COOKBOOK

■ **지역성** - 공간적 지역성(spatial locality)과 시간적 지역성(temporal locality)으로 나뉜다.

- 공간적 지역성 - 근접해 있는 다른 자료들이 짧은 시간 동안 사용되는 것
- 시간적 지역성 - 같은 자료가 짧은 시간 동안 여러 번 사용되는 것
- [그림 11-3]의 행렬의 곱 프로그램을 가지고 공간적 지역성을 살펴보자. 앞에서도 설명했듯이 언어별로 행 우선과 열 우선 에 따라 원소에 접근하는 방법이 완전히 다르다.

```
FOR i= 1 TO N DO
  FOR j= 1 TO N DO
    FOR k= 1 TO N DO
        C[i, j] = C[i, j] + A[i, k] * B[k, j]
    ENDFOR
  ENDFOR
ENDFOR
```

그림 11-3 행렬의 곱 프로그램

11.1 병렬 처리

- [그림 11-3]의 프로그램에서 계산하는 행렬의 곱 C[i, j]는 A의 i번째 행과 B의 j번째 열을 곱하는 것으로, 이를 [그림 11-4]에 나타냈다.

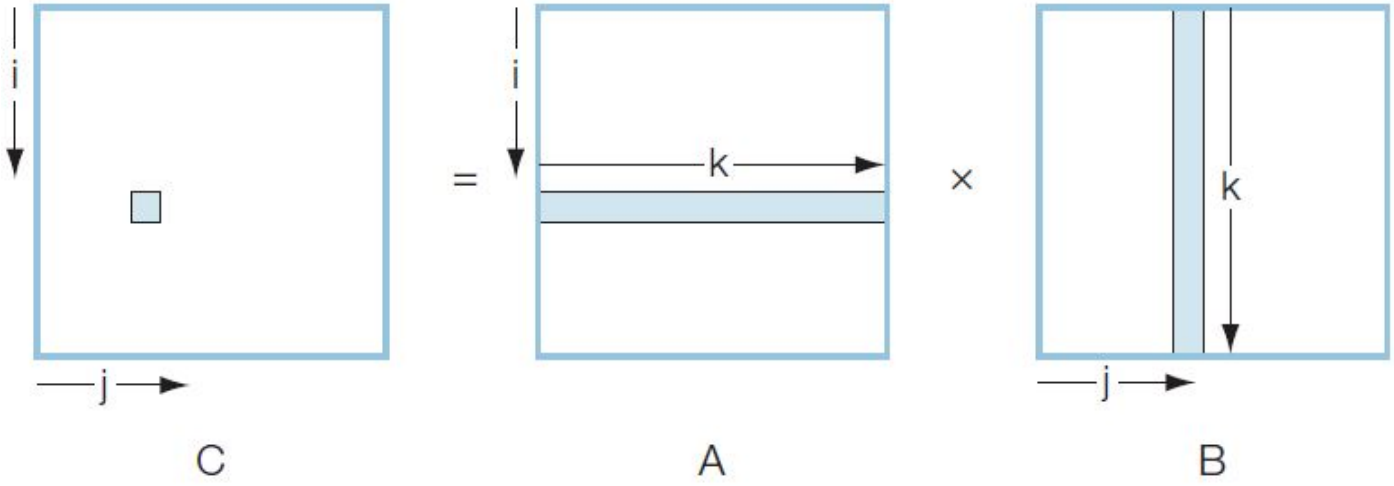


그림 11-4 행렬의 곱 C[i, j]

- 만약 행렬 A와 B가 포트란과 같이 열 우선으로 저장되어 있다면
  - 행렬 계산은 처음에 A[1, 1] 과 B[1, 1] 참조, 다음에 A[1, 2]와 B[2, 1] 참조, 이어서 A[1, 3]과 B[3, 1] 참조, 그리고 계속해서 같은 방법으로 참조한다.
  - 여기서 행렬 A는 캐시 메모리에서 보면 A의 한 워드를 참조한 경우 A의 원소 중 하나만 적중하고(hit) 나머지는 필요 없으므로 매우 비효율적이다.
  - 반면에 행렬 B는 좋은 공간 지역성을 보이면서 연속적으로 처리한다.

11.1 병렬 처리

IT COOKBOOK

- 만약 행렬 A와 B가 C와 같이 행 우선으로 저장되어 있다면
  - 행렬 계산은 처음에는 A[1, 1]과 B[1, 1] 참조, 다음에 A[1, 2]와 B[2, 1] 참조, 이어서 A[1, 3]과 B[3, 1] 참조, 그리고 계속 해서 같은 방법으로 참조한다.
  - 여기서 행렬 A는 좋은 공간 지역성을 보이면서 연속적으로 처리하지만, 행렬 B는 캐시 메모리에서 보면 B의 한 워드를 참조한 경우 B의 원소 중 하나만 적중하고 나머지는 필요 없으므로 매우 비효율적인 방법이다.
- 이번에는 [그림 11-3]의 루프에서 j와 k 제어 변수의 순서를 재순서화(reordering)한 프로그램인 [그림 11-5]를 살펴보자.

```
FOR i = 1 TO N DO
    FOR k = 1 TO N DO
        FOR j = 1 TO N DO
            C[i, j] = C[i, j] + A[i, k] * B[k, j]
        ENDFOR
    ENDFOR
ENDFOR
```

그림 11-5 재순서화 프로그램

11.1 병렬 처리

- [그림 11-5]와 [그림 11-3]의 프로그램이 같은 결과가 나온다는 것을 보여야 하지만 증명은 생략한다.
- 다음으로 [그림 11-5]의 프로그램에서 앞서 설명한 바와 같이 각 요소가 어떻게 메모리를 참조하는지 살펴보자.
- 만약 행렬 A와 B가 열 우선으로 저장되어 있다면
  - 행렬 계산은 처음에 A[1, 1]과 B[1, 1] 참조, 다음에 A[1, 1]과 B[1, 2] 참조, 이어서 A[1, 1]과 B[1, 3] 참조, 그리고 계속해서 같은 방법으로 참조한다. 이를 그림으로 표현하면 [그림 11-6]과 같다. 여기서 행렬 A는 한 원소만 참조하여 계속 사용하므로 참조 시간을 줄일 수 있어 매우 효율적이지만 행렬 B는 열 우선으로 저장되어 있으므로 각 원소들을 참조하는 시간이 많이 걸려 매우 비효율적이다.

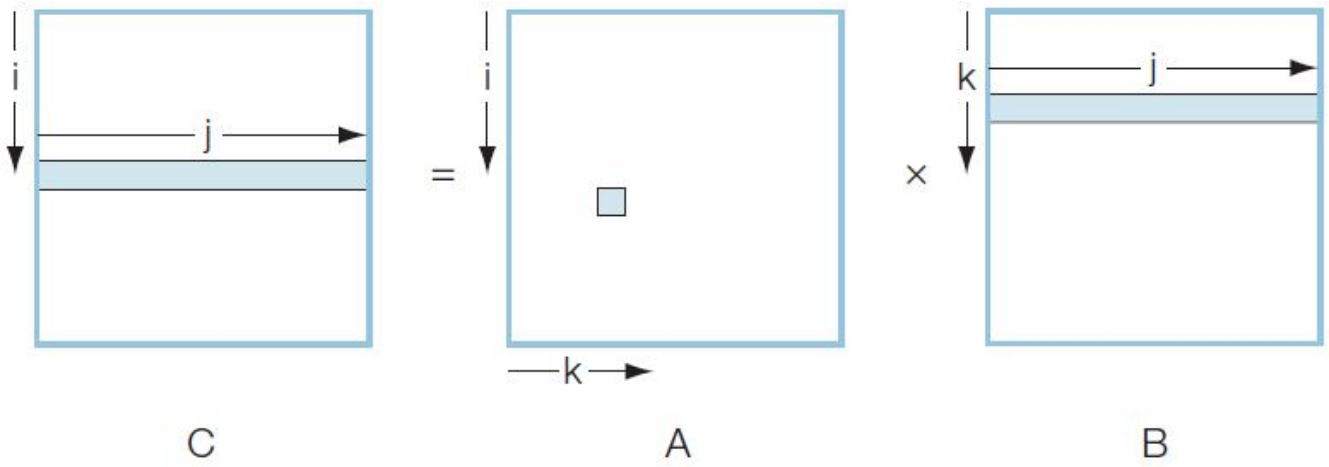


그림 11-6 A의 i번째 행과 B의 j번째 열을 곱한 C[i, j]

11.1 병렬 처리

- 만약 행렬 A와 B가 행 우선으로 저장되어 있다면
  - 행렬 계산은 처음에 A[1, 1]과 B[1, 1] 참조, 다음에 A[1, 1]과 B[1, 2] 참조, 이어서 A[1, 1]과 B[1, 3] 참조, 그리고 계속해서 같은 방법으로 참조한다. 여기서 행렬 A는 한 원소만 참조하여 계속 사용하므로 매우 효율적이고, 행렬 B는 좋은 공간 지역성을 보이면서 연속적으로 처리하므로 매우 좋은 효율을 보인다. 결과적으로 원소들이 행 우선으로 저장되어 있다면 [그림 11-5]와 같이 재순서화된 프로그래밍이 가장 효율적이다.
- 지금까지 설명한 내용을 공간적 지역성이라 한다. 공간적 지역성의 경우, 사용되는 모든 원소가 하나의 캐시 라인상에 있을 때 캐시 라인에서 한 원소를 사용하면 같은 라인상의 모든 원소가 캐시로 올라오므로, 짧은 시간 내에 이러한 원소들을 사용한다면 계속 캐시 내에 존 재하는 원소들을 사용할 수 있어 매우 효율적이다. 이처럼 공간적 지역성이 존재하면 캐시 실패(cache miss)가 줄어들고 프로그램의 속도가 크게 향상된다.
- 한편 시간적 지역성은 같은 자료를 짧은 시간 동안에 여러 번 사용하는 방법이다. 그러므로 캐시의 크기가 크지 않은 경우, 시간적 지역성을 개선하기 위해 2차원 행렬의 곱에서 행렬을 [그림 11-7]과 같이 부분 행렬로 작게 나눌 수 있다면 같은 원소를 여러 번 참조하기 때문에 효율이 좋아질 것이다.

11.1 병렬 처리

IT COOKBOOK

```
FOR IT= 1 TO N BY S DO
    FOR KT= 1 TO N BY S DO
        FOR JT= 1 TO N BY S DO
            FOR I= IT TO MIN(IT+S-1,N) DO
                FOR K= KT TO MIN(KT+S-1,N) DO
                    FOR J= JT TO MIN(JT+S-1,N) DO
                        C[I, J] = C[I, J] + A[I, K] * B[K, J]
                    ENDFOR
                ENDFOR
            ENDFOR
        ENDFOR
    ENDFOR
ENDFOR
```

그림 11-7 **시간적 지역성을 위한 부분 행렬**

- 타일링(tiling)이라는 프로그램 변환으로 부분 행렬을 만들 수 있는데 이는 뒤에서 설명할 것이다

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- 병렬성과 지역성을 위한 최적화를 이루려면 원래 프로그램의 연산 수행 순서를 바꿔야 한다. 다른 최적화 방법과 마찬가지로 프로그램의 출력 값이 변경되지 않는 경우에만 연산의 순서를 바꿀 수 있다. 루프가 병렬화될 수 있는지를 판단하려면 각 루프 경계에 자료 종속이 걸쳐 있는지 살펴보아야 한다. 이 절에서는 자료 종속 및 자료 종속을 계산하기 위한 자료 종속 분석에 대해 알아보자.
- **자료 종속**
  - 다음과 같은 프로그램을 살펴보자.

```
① AA = 10
② BB = AA
③ CC = AA * DD
④ CC = BB
⑤ DD = 20
```

그림 11-8 종속성 프로그램

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- ②번 문장에서 AA를 사용하므로 AA의 정의가 있는 ①번 문장을 실행한 다음 ②번 문장이 실행되어야 한다.
- 마찬가지로 ③번 문장에서도 AA를 사용하므로 ①번 문장을 실행한 다음 ③번 문장이 실행되어야 한다.
- 또한 ③번 문장에서 DD를 사용하고 ⑤번 문장에서 DD를 정의하므로 ⑤번 문장을 먼저 실행하고 ③번 문장을 실행하면 결과 값이 다를 수 있다. 그러므로 ③번 문장을 실행한 다음 ⑤번 문장을 실행해야 한다.
- ③번 문장에서 CC를 정의하고 ④번 문장에서 다시 CC를 정의한다. 그리고 ②번 문장에서 BB를 정의하고 ④번 문장에서 사용하므로 ②번 문장을 실행한 다음 ④번 문장을 실행해야 한다.

■ 이러한 정보를 표현하기 위해 자료 종속 그래프(data dependence graph)를 이용

- 자료 종속 그래프는 다중 그래프인데, 다중 그래프는 2개의 노드가 여러 개의 간선과 연결되어 있는 것을 말한다. 여기서 노드는 문장 혹은 기본 블록을 의미하며, 간선은 문장의 재순서화가 가능하지 않다는 것을 보여준다.

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- 종속에는 자료 종속과 제어 종속(control dependence)이 있다.
  - 자료 종속은 다시 참 종속(true dependence)인 흐름 종속(flow dependence) 그리고 거짓 종속(false dependence)인 반 종속(anti-dependence)과 출력 종속(output dependence)의 세 가지 유형이 있다.
  - 흐름 종속은 '정의' 후에 '사용'으로, 하나의 변수가 '정의'되어 있고 연속적으로 실행된 문장 에서 '사용'되는 경우이다. 즉 두 문장 Si와 Sj가 있을 때 Si에서 변수 X가 '정의'되고 Sj에서 X 가 '사용'되며 Si가 Sj 이전에 수행되는 경우이고, 이를 Si δ Sj로 나타낸다.
  - 거짓 종속은 종속이 제거될 수 있는 것인데 이를 재명명(renaming) 기법으로 제거한다.
  - 반 종속은 '사용' 후에 '정의' 로, 하나의 변수가 한 문장 안에서 '사용'되고 연속적으로 실행되는 문장에서 '정의'되는 경우 이다. 즉 문장 Si에서 X를 '사용'하고 Sj에서 X가 '정의' 되며 Si가 Sj 이전에 수행되는 경우이고, 이를 Si δ^a Sj로 나타낸다.
  - 출력 종속은 '정의' 후에 '정의'로, 하나 의 변수가 한 문장 안에서 '정의'되고 연속적으로 실행되는 문장에 서 다시 '정의'되는 경우이다. 즉 두 문장 Si와 Sj에서 같은 변수가 선언되고 Si와 Sj 이전에 수행되는 경우이며, 이를 Si δ^o Sj로 나타낸다.
  - [그림 11-8]을 자료 종속 그래프로 나타내면 [그림 11-9]와 같다. S1과 S2, S1과 S3, S2와 S4는 흐름 종속이고 S3과 S4는 출력 종속, S3 과 S5는 반 종속이므로 S1 δ S2, S1 δ S3, S2 δ S4, S3 δo S4, S3 δa S5 이다.

11.2 자료 종속과 자료 종속 분석

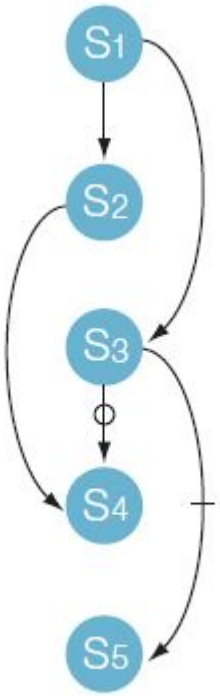


그림 11-9 자료 종속 그래프

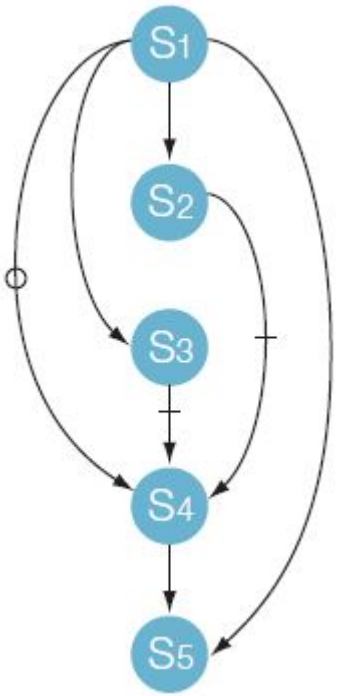
[예제 11-2] 자료 종속 그래프 그리기

- 다음 프로그램에 대한 자료 종속 그래프를 그려보자.

```
    for i =1 to N do
S1 : A[i + 3] = ...
S2 : ... = A[i + 1]
S3 : ... = A[i + 2]
S4 : A[i] = ...
S5 : ... = A[i - 1]
    endfor
```

- [풀이] 자료 종속을 분석하면 $S_1\ \delta\ S_2$, $S_1\ \delta\ S_3$, $S_1\ \delta\ S_5$, $S_1\ \delta^o\ S_4$, $S_2\ \delta^a\ S_4$, $S_3\ \delta^a\ S_4$, $S_4\ \delta\ S_5$임. 이를 자료 종속 그래프로 나타내면 다음과 같다

11.2 자료 종속과 자료 종속 분석



11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- 종속 그래프는 비순환 그래프(acyclic graph)이며, 이런 그래프를 때때로 우선순위 그래프(precedence graph)라고 한다.
- 이제 거짓 종속을 제거해보자.
  - [그림 11-8]의 자료 종속 그래프에서 $S_3\ \delta^o\ S_4$, $S_3\ \delta^a\ S_5$이다.
  - 출력 종속을 제거하려면 다른 이름으로 대체하는 재명명 방법을 사용한다.
  - 따라서 ④번 문장의 CC를 CCC로 변환하면 ③번과 ④번 문장에서 출력 종속이 없어진다.
  - 마찬가지로 ③번 문장과 ⑤번 문장 사이에 반 종속이 존재하므로 ⑤번 문장의 DD를 DDD로 변환한다.
- 이렇게 [그림 11-8]의 프로그램에서 출력 종속과 반 종속을 제거한 것은 [그림 11-10]과 같다. 그리고 [그림 11-10]의 프로그램에 대한 자료 종속 그래프를 [그림 11-11]에 나타냈으며, 그림을 통해 출력 종속과 반 종속이 제거되고 흐름 종속만 남았음을 확인할 수 있다.

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

```
① AA = 10
② BB = AA
③ CC = AA * DD
④ CCC = BB
⑤ DDD = 20
```

그림 11-10 출력 종속과 반종속이 제거된 프로그램

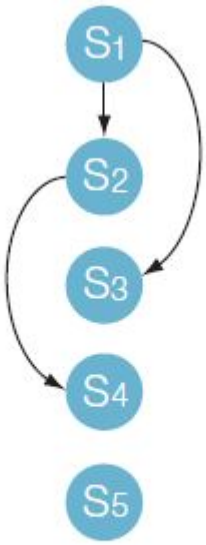


그림 11-11 출력 종속과 반종속이 제거된 자료 종속 그래프

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- 한 루프에서 2개의 다른 반복(iteration) 사이에 2개의 문장이 자료 종속을 가진 경우를 **루프 캐리드 종속**(loop carried dependence)이라 하고, 한 루프의 같은 반복에서 2개의 문장 사이에 자료 종속을 가진 경우를 **루프 독립 종속**(loop independent dependence)이라 한다.
- **[예제 11-3] 루프 캐리드 종속 구하기**
  - 다음 프로그램에서 루프 캐리드 종속을 구해보자.

```
① for i = 1 to n do
②   for j = 1 to m do
③       A[i, j] = ...
④           ... = A[i, j]
⑤       B[i, j+1] = ...
⑥           ... = B[i, j]
⑦       C[i+1, J] = ...
⑧           ... = C[i, j+1]
⑨   endfor
⑩ endfor
```

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- [풀이]
- ⑤번과 ⑥번 문장을 보면 j에 대해 루프 캐리드 종속이 존재한다. 왜냐하면 루프 캐리드 종속은 한 루프에서 2개의 다른 반복 사이에 2개의 문장이 자료 종속을 가진 경우이므로, j 반복에 대해 살펴 보면 j = 1인 경우 ⑤번과 ⑥번 문장은 종속이 존재하지 않는다. 하지만 j = 2인 경우 ⑤번 문장에서 는 j = 1일 때 B[i, j+1]이 B[i, 2]가 되고, ⑥번 문장에서는 j = 2일 때 B[i, j]는 B[i, 2]가 된다. 즉 ⑤번 문장의 j = 1일 때 B[i, 2]와 ⑥번 문장의 j = 2일 때 B[i, 2]가 자료 종속을 갖는다. 이런 경우를 루프 캐리드 종속이 존재한다고 한다.
- 반면에 ③번과 ④번 문장은i = 5, j = 5일 때 둘 다 A[5, 5] 를 가지므로 자료 종속이 발생하는데, 이를 루프 독립 종속이 존재한다고 한다.

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

제어 종속성(control dependence)

- 한 문장을 실행할 때 다른 문 장에 의해 생성된 값에 의존하여 실행될 수도 있고 그렇지 않을 수도 있는 경우를 제어 종속성이 있다고 한다.
- if-else 제어문에서 조건에 따라 then 이하를 실행하거나 else 이하를 실행하면 제어 종속성이 있다. 아래에서 S_1과 S_2는 cond에 제어 종속적이다.

  if cond S_1; else S_2;

- **[예제 11-4] 제어 종속 구하기**
- 다음 프로그램에서 제어 종속을 구해보자.

  S_1 : a = b + c;

  S_2 : b = a * 3;

  S_3 : a = 2 * c;

  S_4 : if (b > = 0)

  S_5 : then d = 1

  S_6 : else d = 0

  fi

- S_4와 S_5 사이에 제어 종속성이 존재하고, S_4와 S_6 사이에도 제어 종속성이 존재한다.

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

■ 자료 종속 분석

- 자료 종속이란
  - 같은 메모리 장소를 사용하는 두 문장 사이의 수행 순서가 유지되어야 하는 성질
  - 즉 두 문장 사이에 종속성이 없을 경우 두 문장은 병렬로 수행이 가능하다.
- 자료 종속을 찾으려면
  - 등식과 부등식으로 표현된 여러 제약을 만족하는 정수해가 존재하는지 판별
  - 등식은 참조를 나타내는 행렬과 벡터에서 얻을 수 있고, 부등식은 루프 경계로부터 유도해낼 수 있다.
  - 한편 등식은 부등식으로 나타낼 수도 있다.
    - 예컨대 등식 $x = y$는 2개의 부등식 $x \geqq y$와 $x \leqq y$로 나타낼 수 있다.
- 따라서 자료 종속성은 선형 부등식 집합을 만족하는 정수 해를 찾는 문제
  - 이러한 문제를 정수 선형 프로그래밍(integer linear programming)이라 한다.
  - 정수 선형 프로그래밍은 NP 완전(NP-complete) 문제이다.
    - 즉 자료 종속성 문제는 정수 선형 프로그래밍 문제를 해결해야 하지만 이 문제는 NP 완전 문제이므로 풀기가 어렵다. 따라서 경험적인 방법을 동원하여 정수 선형 프로그래밍 문제를 해결하며, 경험적 알고리즘은 대부분의 경우에 꽤 빠르게 동작한다.

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- 1절에서 다루었던 다음 예를 살펴보자.

```
for I = 1 to 100 do
  A(I) = B(I) * C(I)
  B(I) = C(I) / A(I-3)
Endfor
```

- 여기서 A라는 변수에 흐름 종속이 있으므로 순차적인 실행을 한다. 하지만 첨자 변수 때문에 흐름 종속이 항상 존재하는 것은 아니다. 이 문장이 어떻게 실행되는지 제어 변수를 순차적으로 적용해보자.

```
I = 1     A(1) = B(1) * C(1)
          B(1) = C(1) / A(-2)
I = 2     A(2) = B(2) * C(2)
          B(2) = C(2) / A(-1)
I = 3     A(3) = B(3) * C(3)
          B(3) = C(3) / A(0)
I = 4     A(4) = B(4) * C(4)
          B(4) = C(4) / A(1)
I = 5     A(5) = B(5) * C(5)
          B(5) = C(5) / A(2)
```

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

```
I = 6      A(6) = B(6) * C(6)
           B(6) = C(6) / A(3)
I = 7      A(7) = B(7) * C(7)
           B(7) = C(7) / A(4)
I = 8      A(8) = B(8) * C(8)
           B(8) = C(8) / A(5)
I = 9      A(9) = B(9) * C(9)
           B(9) = C(9) / A(6)
I = 10     A(10) = B(10) * C(10)
           B(10) = C(10) / A(7)
  ⋮
I = 96     A(96) = B(96) * C(96)
           B(96) = C(96) / A(93)
I = 97     A(97) = B(97) * C(97)
           B(97) = C(97) / A(94)
I = 98     A(98) = B(98) * C(98)
           B(98) = C(98) / A(95)
I = 99     A(99) = B(99) * C(99)
           B(99) = C(99) / A(96)
I = 100    A(100) = B(100) * C(100)
           B(100) = C(100) / A(97)
```

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- I가 1일 때와 2일 때 4개의 문장 사이에는 종속성이 존재하지 않는다. 따라서 4개의 문장을 병렬로 처리해도 정확한 해를 구할 수 있다.
- 또한 I가 1일 때와 2일 때, 그리고 3일 때 6개의 문장 사이에도 종속성이 존재하지 않으므로 6개의 문장을 병렬로 처리해도 될 것이다.
- 다시 확장해보자. I가 1일 때, 2일 때, 3일 때, 4일 때 8개의 문장 사이에는 종속성이 존재한다. I가 4일 때와 1일 때 처음으로 흐름 종속이 생긴다.
- 그러므로 이 프로그램에서는 먼저 I가 1, 2, 3일 때 6개의 문장을 동시에 병렬로 처리하고, 다음으로 I가 4, 5, 6일 때 6개의 문장을 동시에 병렬로 처리할 수 있다.
- 이렇게 처리하는 경우 이에 대한 흐름 종속은 [그림 11-12] 와 같이 나타낼 수 있다.

11.2 자료 종속과 자료 종속 분석

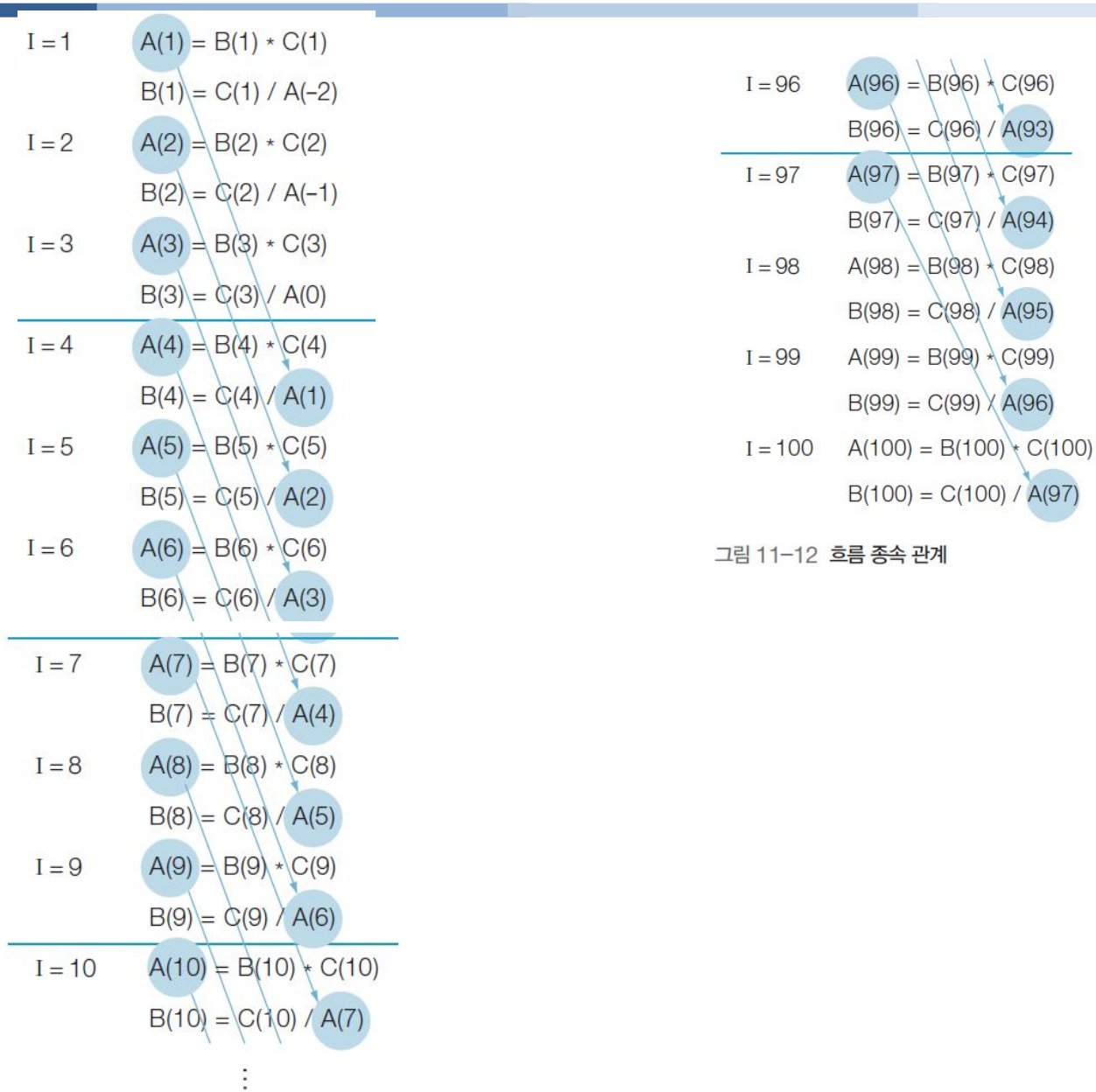


그림 11-12 흐름 종속 관계

11.2 자료 종속과 자료 종속 분석

- 원래 프로그램에서 첨자 변수인 I와 I-3이 같아지는 값이 I=4, 7, 10, ...일 때를 찾아내야 할 것이다.
- 수학식으로만 본다면 방정식 I=I-3을 푸는 것처럼 보이지만 이 종속도 앞에서 설명한 루프 캐리드 종속이다. I가 같은 값이 아니라 I가 1일 때와 4일 때같이 서로 다른 I 값을 구하는 문제이므로 방정식은 I=I-3이 아니라 J=K-3이다.
- 그래서 방 정식 J=K-3을 풀면 K가 4일 때 J는 1이 될 수 있다. 원래 프로그램의 제어 변수에서 I가 1일 때와 4일 때 만나는 것이다.
- 이제 방정식 J=K-3을 풀어야 한다.
  - 2원 1차 방정식이므로 일반 해가 존재하려면 방정식이 2개 있어야 한다.
  - 그런데 방정식이 하나이고 미지수는 2개인 방정식으로, 이는 힐버트의 난제 23개 중 하나이다.
  - 이 방정식은 일반 해가 존재하지 않고 특수해가 존재하는 경우이다. 특수해는 해가 존재하는 경우 해가 많은 부정해이다. 그러므로 이 방정식도 특수해가 존재한다면 부정해이다.
  - 일반 해가 존재하지 않는 방정식 J=K-3과 같은 방정식은 선형 디오판토스 방정식(linear Diophantine equations) 이론을 이용하여 방정식의 정수해가 존재하는지 판별하기 위해 최대 공약 수(greatest common divisor, GCD)를 검사한다.
  - 여기서 정수해가 존재하지 않으면 자료 종속성이 없으며, 정수해가 존재하면 유클리드 알고리즘의 해를 백 워드(backward) 방법으로 구할 수 있다.

11.2 자료 종속과 자료 종속 분석

IT COOKBOOK

- 그렇다면 종속 검사는 어떻게 해야 할까? 종속 검사는 병렬화를 위한 가장 중요한 분석 작업 의 하나로 다양한 검사법이 연구되었다.
- 종속 분석 방법 중 루프 영역과 상관없이 종속 방정식이 정수 해를 갖는지를 판단하는 GCD 검사 방법에 대해 살펴보자. 첫 번째는 방정식의 정 수해가 존재하는지 검사하는 것이다.

- **[정리 11-1]** a, b, c를 양의 정수라 하고 디오판토스 방정식 ax + by = c가 정수 해를 가질 필요 충분조건은 GCD(a, b) | c이다.
- **[증명] 생략**

- **[예제 11-5] 정수해 존재 여부 검사하기 1**
  - [그림 11-12]에서 정수 해를 갖는지 알아보자.
  - [풀이] 방정식 j=k-3에 대한 정수 해를 구하는 것이다. 정수해가 존재하려면 [정리 11-1]에 의해 GCD (a, b) | c여야 하는데 GCD(1, 1) | 3이므로 3이 1로 나눠 떨어진다. 따라서 방정식의 해가 존재한다.

11.3 루프 변환

- 루프 변환(loop transformation)은 문장 및 루프 반복의 실행 순서를 재순서화 하는 데 사용된다. 자료 종속은 자료 종속 거리(data dependence distance)에 따라 균일(uniform) 종속 거리와 불균일(non-uniform) 종속 거리로 나눌 수 있다.
- 만약 하나의 루프 첨자 변수로만 구성된 첨자식이 각각 ai + b, ci + d(a, b, c, d는 정수, i 는 루프 변수)인 경우, 첨자식에서 a = c이면 | (ai±b) - (ci±d) | 인 균일 종속 거리가 존재 하고 a≠c이면 불균일 종속 거리가 존재한다. 균일 종속 거리의 경우 하나의 종속 거리만 존재하고, 불균일 종속 거리의 경우 여러 개의 종속 거리가 존재한다.
- 이 절에서는 루프 재구조화(loop restructuring) 방법으로 루프 융합, 루프 분리, 루프 역순, 루프 교 환, 루프 기울임, 단일 모듈형 변환, 스트립 마이닝, 루프 타일링 등에 대해 살펴보자

11.3 루프 변환

IT COOKBOOK

■ 루프 융합

- **루프 융합**(loop fusion)은 큰 크기 병렬화가 목적인 변환에서 이용된다. 루프에서 다수의 중첩 루프를 가진 문장이 이웃한 루프의 헤더와 같은 경우 이웃한 루프를 병합하는 변환 방법이다. 단, 루프 변수의 영역이 동일하고 루프 융합 후 자료 종속 관계가 변하지 않아야 한다.
- 예를 들어 다음 프로그램을 살펴보자.

```
for I = 1 to n do
    A(I) = B(I) * C(I)
endfor
for I = 1 to n do
    B(I) = C(I) / A(I-3)
endfor
```

- 이 프로그램은 루프의 헤더가 같으므로 다음과 같이 루프 융합을 할 수 있다.

```
for I = 1 to n do
    A(I) = B(I) * C(I)
    B(I) = C(I) / A(I-3)
endfor
```

IT COOKBOOK

[예제 11-7] 루프 융합하기

- 다음 루프에 대해 루프 융합을 해보자.

```
for k = 1 to kmaxd do
  for j = 1 to jmaxd do
     f(m, j, k) = f(m, j, k) * b(m)
  Endfor
Endfor
for k = 1 to kmaxd do
  for j = 1 to jmaxd do
    for i = 2 to n-1 do
       f(m, j, k) = (f(m, j, k) * a(m) * f(m, j, k) * b(m)
    endfor
  Endfor
Endfor
```

11.3 루프 변환

IT COOKBOOK

```
for k = 1 to kmaxd do
    for j = 1 to jmaxd do
        tot(j, k) = 0
    Endfor
  Endfor
  for k = 1 to kmaxd do
    for j = 1 to jmaxd do
      for i = 2 to n-1 do
          tot(j, k) = tot(j, k) + d(m) * f(m, j, k)
      endfor
    Endfor
  endfor
```

11.3 루프 변환

IT COOKBOOK

- **[풀이]** 헤더가 같은 것에 대해 루프 융합을 한다.

```
for k = 1 to kmaxd do
  for j = 1 to jmaxd do
    f(m, j, k) = f(m, j, k) * b(m)
    for i = 2 to n-1 do
      f(m, j, k) = (f(m, j, k) * a(m) * f(m, j, k) * b(m)
    endfor
    tot(j, k) = 0
    for i = 2 to n-1 do
      tot(j, k) = tot(j, k) + d(m) * f(m, j, k)
    endfor
  endfor
endfor
```

- 이 코드는 자료 종속을 계산하면 융합이 좀 더 가능할 것이다.

■ 루프 융합은 불필요한 장벽 동기화(barrier synchronization)와 반복 오버헤드를 줄이는 변환 방법. 일반 프로그램에서 프로그램 효율을 높이기 위해 사용할 수도 있다.

11.3 루프 변환

IT COOKBOOK

■ 루프 분리

- 루프 안의 문장을 병렬화할 수 없는 경우, 또는 병렬화를 위해 부분적으로 루프를 만들거나 자료 종속을 분리하기 위해 루프 문장을 나누는 것을 **루프 분리**(loop distribution, loop fission)라고 한다.
- 루프 분리를 하는 과정은 루프 융합과 반대이다. 루프를 분리하려면 자료 종속 그래프를 그린 다음, 서로 다른 문장의 같은 루프 첨자를 다른 루프 첨자로 바꾸어 코드로 변환한다.
- 예를 들어 다음 프로그램을 살펴보자.

```
for I = 1 to n do
    A(I) = B(I) * C(I)
    B(I) = C(I) / D(I)
endfor
```

- 위의 프로그램에 대해 루프 분리를 하면 다음과 같다.

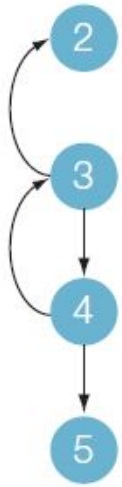
```
for k = 1 to n do
    A(k) = B(k) * C(k)
endfor
for m = 1 to n do
    B(m) = C(m) / D(m)
endfor
```

[예제 11-8] 루프 분리하기

- 다음 루프에 대해 루프 분리를 해보자.

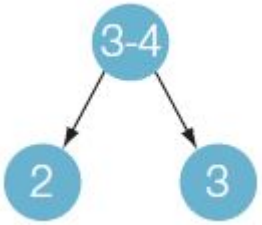
```
① for I = 1 to n do
②     A[I] = A[I] + B[I-1]
③     B[I] = C[I-1] * Y + K
④     C[I] = B[I]
⑤     D[I] = C[I]
⑥ endfor
```

- 이 루프에 대한 종속 그래프는 다음과 같다.



11.3 루프 변환

- ③번과 ④번 행은 강 결합(strongly connected)이므로 같은 루프에서 유지해야 한다. 이렇게 해서 루프 분리는 {2}, {3,4}, {5}와 같이 분리된 루프 3개로 만들 수 있다. 비순환 그래프는 오른쪽과 같다.



- 따라서 루프 분리의 결과는 다음과 같다.

```
① for Ib = 0 to n-1 do
③     B[Ib+1] = C[Ib] * Y + K
④     C[Ib+1] = B[Ib+1]
⑥ endfor
① for Ib = 0 to n-1 do
②     A[Ib+1] = A[Ib+1] + B[Ib]
⑥ endfor
① for Ib = 0 to n-1 do
⑤     D[Ib+1] = C[Ib+1]
⑥ endfor
① I = n+1
```

- 하나의 배열은 메모리에서 연속된 장소를 사용할 가능성이 매우 높기 때문에 캐시와 같은 기억장치의 효율을 높일 수 있다.

11.3 루프 변환

■ 루프 역순

- 컴파일러는 역으로 루프를 수행할 수 있는데 이것을 **루프 역순(**loop reversal)이라 한다. 모든 반복문은 루프 역순이 가능하다. 만약 순차적 루프가 종속성 관계를 가진다면 루프 역순의 종속은 역방향이 되고 종속 관계에 위배된다. 따라서 루프 역순은 종속 관계가 없을 때만 가능하다. 루프 역순을 사용하면 2개의 루프를 융합하는 루프 융합이 가능하다.
- 예를 들어 다음 프로그램을 살펴보자.

```
for k = 0 to n do
    A(n-k) = B(k) * C(k)
endfor
for j = 0 to n do
    D(j) = A(j)
endfor
```

- 위의 프로그램에 대해 루프 역순을 하면 다음과 같다.

```
for p = 0 to n do
    A(p) = B(n-p) * C(n-p)
    D(p) = A(p)
endfor
```

[예제 11-9] 루프 역순하기

- 다음 루프에 대해 루프 역순을 해보자.

```
for i = 1 to n do
    a[i] = b[i] + 1
    c[i] = a[i] / 2
endfor
for i = 1 to n do
    d[i] = 1 / c[i+1]
Endfor
```

- [풀이] 루프 역순은 다음과 같다.

```
for i = n downto 1 do
    a[i] = b[i] + 1
    c[i] = a[i] / 2
    d[i] = 1 / c[i+1]
endfor
```

11.3 루프 변환

IT COOKBOOK

■ 루프 교환

- 다중으로 내포된 루프 구조에서 자료 종속 관계의 특성만 유지된다면 루프의 순서를 교환해도 결과에는 영향이 없다. 즉 여러 개의 중첩 루프가 있을 때 바깥 루프와 안쪽 루프를 바꾸는 것인데 이를 **루프 교환**(loop interchange)이라 한다.
- 예를 들어 다음 프로그램을 살펴보자.

```
SUBROUTINE P
  REAL A(N, N)
  CALL S(A)
SUBROUTINE S(F)
  REAL F(N, N)
  INTEGER I, J
  for J = 1 to N1
    for I = 1 to N2
      F(J, I) = F(J, I-1) + 10
    endfor
  endfor
```

- 여기서 I 루프와 J 루프를 교환해도 자료 종속 관계가 그대로 유지되므로 2개의 루프를 교환할 수 있다.

11.3 루프 변환


```
SUBROUTINE P
  REAL A(N, N)
  CALL S(A)
SUBROUTINE S(F)
  REAL F(N, N)
  INTEGER I, J
  for I = 1 to N1
    for J = 1 to N2
        F(J, I) = F(J, I-1) + 10
    endfor
  endfor
```

- I 루프 사이에는 루프 캐리드 종속 관계가 있으나 J 루프 사이에는 루프 캐리드 종속 관계가 없으므로 J 루프를 병렬화할 수 있다. 병렬화를 만들기 위해, 그리고 병렬성을 증가시키기 위해 루프에서 중첩 루프 헤더를 교환한다. 병렬화를 위해 두 루프를 교환한 다음 밖으로 옮겨진 J 루프를 병렬화하면 된다.
- 또한, 이미 앞에서 설명한 거와 같이 행 우선과 열 우선일 때도 루프 교환하기를 했다.

IT COOKBOOK

[예제 11-10] 루프 교환하기

- 다음 루프에 대해 루프 교환을 해보자.

```
for J = 1 to n do
  for I = 1 to n do
     A[I, J] = A[I, J-1] + B[I, J]
  endfor
endfor
```

- [풀이] 루프 교환을 하면 다음과 같다.

```
for I = 1 to n do
  for J = 1 to n do
     A[I, J] = A[I, J-1] + B[I, J]
  endfor
endfor
```

루프 기울임

- 루프 내 반복 공간(iteration space)의 모양을 바꿈으로써 반복 간의 종속성을 반복 내의 종속성으로 바꿀 수 있는데 이를 **루프 기울임**(loop skewing)이라 한다. 일부 루프 기울임은 루프 교환을 가능하게 한다.
- 예를 들어 다음 프로그램을 살펴보자.

```
for I = 1 to n+m-1 do
  for J = max(1, I+n) to min(I, m) do
    A[I, J] = A[I-1, J-1]
  Endfor
Endfor
```

- 위의 프로그램에 대해 루프 기울임을 하면 다음과 같다.

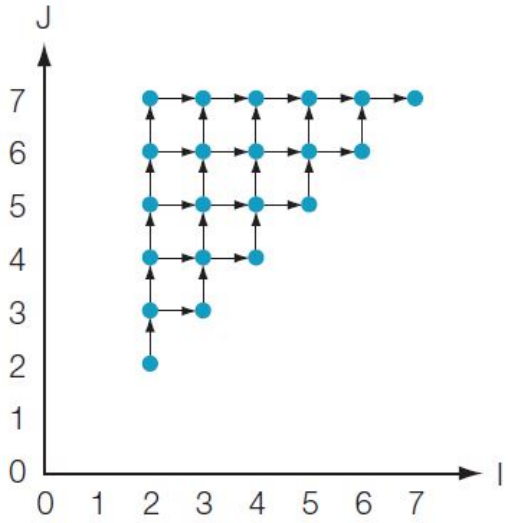
```
for p = 1 to n do
  for q = 1 to m do
    A[p, q-p] = A[p-1, q-p-1]
  Endfor
endfor
```

[예제 11-11] 루프 기울임하기

- 다음 루프의 반복 공간을 루프 기울임 해보자.

```
for I = 2 to n do
  for J = I to n do
    A[I, J] = A[I, J-1] + A[I-1, J]
  Endfor
endfor
```

- [풀이] 루프의 반복 공간을 그림으로 나타내면 오른쪽과 같다.

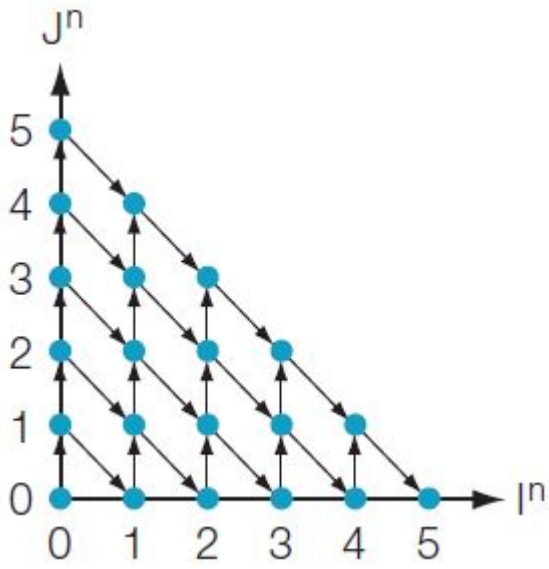


11.3 루프 변환

- 이 프로그램을 루프 기울임 해보자.

```
for J = 2 to n do
  for I = 2 to J do
     A[I, J] = A[I, J-1] + A[I-1, J]
  Endfor
Endfor
```

- 이에 대한 반복 공간을 그림으로 나타내면 다음과 같다.



11.3 루프 변환

- 앞에서 설명한 루프 역순, 루프 교환, 루프 기울임은 선형 루프 변형(linear loop transformation)으로 통합하여 변환할 수 있는데 이를 **단일 모듈형 변환**(unimodular transforms)이라 한다.
- 단일 모듈형 변환은 단일 모듈형 행렬(행렬식 = ±1)을 사용하여 중첩 루프를 재구성해서 수행한다.

11.3 루프 변환

■ 스트립 마이닝

- 하나의 루프를 중첩된 루프 구조로 변환하는 것을 **스트립 마이닝**(strip-mining). 변환된 루프의 안쪽 루프에서는 기억장치의 한 블록 단위로 자료를 처리할 수 있도록 변환한다.
- 스트립 마이닝은 하드웨어의 특성과 매우 밀접한 관계가 있다. 예를 들어 캐시의 블록이 64 워드이거나 벡터 프로세서의 크기가 64워드인 경우 64워드씩 처리할 수 있도록 중첩된 루프를 변환한다.
- 예를 들어 다음 프로그램을 살펴보자.

```
for I = 1 to n do
    A[I] = B[I] + C[I]
endfor
```

- 이 루프에 대해 2개의 중첩된 루프로 스트립 마이닝을 해보자. 스트립 마이닝을 하기 위해서는 중첩된 루프로 만들어야 한다. 먼저 외부 루프(outer loop)는 스트립의 크기를 s라고 할 때 s만큼씩 증가하게 만들면 된다. 그리고 내부 루프(inner loop)는 외부 루프의 첨자에 따라 수정하면 된다.

```
for II = 1 to n by s do
  for I = II to min(n, II+s-1) do
    A[I] = B[I] + C[I]
  endfor
endfor
```

[예제 11-12] 스트립 마이닝 하기

- 다음 루프의 스트립 크기가 8일 때 스트립 마이닝을 해보자.

```
for I = 1 to 100 do
    A[I + 3] = A[I] + B[I]
endfor
```

- [풀이] 이 루프에 대해 스트립 마이닝을 하면 다음과 같다.

```
for II = 1 to 100 by 8 do
  for I = II to min(100, II+7) do
      A[I + 3] = A[I] + B[I]
  Endfor
endfor
```

11.3 루프 변환

루프 타일링

- **루프 타일링**(loop tiling)은 전체 지역의 병렬화가 불가능한 경우에 지역을 삼각형, 사각형 등 다각형의 블록 단위나 부분 배열 단위로 나누어 병렬화가 가능하게 하는 변환 방법이다.
- 예를 들어 다음과 같은 루프가 있다.

```
for I = l0 to hi do
```

- 이 루프에 대한 루프 타일링을 위한 공식은 다음과 같다.

```
for It = floor((l0-t0)/ts)*ts+t0 to floor((hi-t0)/ts)*ts+t0 by ts do
  for I = max(l0, It) to min(hi, It+ts-1) do
```

IT COOKBOOK

[예제 11-13] 루프 타일링하기

- 다음 루프에 대해 타일 크기가 20이고 오프셋이 5가 되도록 루프 타일링을 해보자.

```
for I = 1 to 50 do
  for J = I to 60 do
     A[I, J] = A[I, J] + 10
  Endfor
Endfor
```

- 루프 타일링을 하면 다음과 같다.

```
for It = 15 to 45 by 20 do
  for I = max(1, It) to min(50, It+19) do
    for Jt = floor((I-5)/20)*20+5 to 45 by 20 do
      for J = max(I, Jt) to min(60, Jt+19) do
         A [I, J] = A[I, J] + 10
      endfor
    endfor
  endfor
endfor
```

11.3 루프 변환

IT COOKBOOK

- 1970년대와 1980년대 초에 벡터화와 병렬화를 개선하기 위해 루프 융합, 루프 분리, 스트립 마이닝, 루프 교환 등 루프 변환을 사용하기 시작했으며,
- 이와 관련하여 세 가지 주요 프로젝트가 수행되었다.
  - 일리노이대학의 쿡(David J. Kuck) 교수가 이끌었던 Parafrase 프로젝트,
  - 라이스대학의 케네디(Ken Kennedy) 교수가 이 끌었던 PFC 프로젝트,
  - IBM 연구소의 앨런(Frances E. Allen) 박사가 이끌었던 PTRAN 프로젝트
- 이런 세 가지 프로젝트는 병렬 계산(parallel computing)을 위한 병렬 컴파일러를 만드는 것으로 초창기 병렬 컴파일러의 선구자 역할을 했다.

Thank You